

while executing the code, use the SAR (system activity report) from the sysstat package, which is my favourite.

```
sar -u 2 400
```

Here, ‘-u’ is for CPU reporting; ‘2’ implies a 2-second interval; and ‘400’ is to run it 400 times.

Surprisingly, the code eats lots of cycles for kernel-space, as compared to user-space.

Compile the code using the `-pg` option to include the profiler code in the executable, as follows:

```
gcc -pg gprof_test_code_svp.c
```

This will generate an *a.out* executable file. You can specify another output filename with ‘-o’. Now run the binary with *a.out*. After executing the code, the following output will be generated:

```
root@ubuntu:~# gcc -pg gprof_test_code_svp.c
root@ubuntu:~# ./a.out
Initial value in Main function= 0
Last value in Main function (value(p) incremented for) = 10000
Func1(),Func2(),Func3() & filegen() called for 399 399 9601 1197
times
root@ubuntu:~#
```

Flat-profile generation

After successful execution of the program, it will generate the file *gmon.out*. Use Gprof on this file to generate the graph:

```
gprof -p a.out gmon.out
```

```
Flat profile:
Each sample counts as 0.01 seconds.
%   cumulative self self total
time seconds seconds calls us/call us/call name
97.98 0.97 0.97 9601 101.03 101.03 func3
2.02 0.99 0.02 399 50.13 50.13 func2
0.00 0.99 0.00 1197 0.00 0.00 filegen
0.00 0.99 0.00 399 0.00 50.13 func1
<output snipped>
```

This generates flat-profile analytics, where time calls are given for comparison. Indications from this profile are:

- The program took 1.46 minutes of clock time to complete. From the flat-profile, *func3* consumed 0.97 seconds per call and was called 9601 times; around 101 milliseconds are spent in this function for every call to *func3*. The same values in the *self us/call* and *total us/call* field indicate that the complete time was spent on the function’s own operations, and not on the children functions.
- In *func1*, which was the last entry presented in the

profile, there is practically no time spent in itself (*self us/call*) but as it calls *func2*, most of its time is spent there. Hence, the *total us/call* field shows 50.13 milliseconds—the same as that of *func2*.

- In *func2*, the *self us/call* and *total us/call* field are the same—the time is spent in doing self operations.
- There is practically no time spent in the *filegen* function according to *Gprof*, as the *md5sum* is a system call, which is not presented in *Gprof*.

So, using this, we can clearly look for bottlenecks present in the code consuming CPU time.

Call-graph profile generation

To generate a call-graph profile of the functions used in the program, use *gprof* with the ‘*q*’ switch:

```
gprof -q a.out gmon.out
```

Call graph (explanation follows)

granularity: each sample hit covers 4 byte(s) for 1.01% of 0.99 seconds

index % time self children called name

```
<spontaneous>
[1] 100.0 0.00 0.00 main [1]
      0.97 0.00 9601/9601 func3 [2]
      0.00 0.02 399/399 func1 [3]
-----
      0.97 0.00 9601/9601 main [1]
[2] 98.0 0.97 0.00 9601 func3 [2]
-----
      0.00 0.02 399/399 main [1]
[3] 2.0 0.00 0.02 399 func1 [3]
      0.02 0.00 399/399 func2 [4]
-----
      0.02 0.00 399/399 func1 [3]
[4] 2.0 0.02 0.00 399 func2 [4]
      0.00 0.00 1197/1197 filegen [5]
-----
      0.00 0.00 1197/1197 func2 [4]
[5] 0.0 0.00 0.00 1197 filegen [5]
-----
```

<Output Snipped>

Indications from call-graph are:

- The number after the name indicates the index numbers. So we can see that main spends 0.99 seconds in the children waiting for *func3* to complete, which is in the *children* field. This is substantiated by the 0.97 seconds spent per call in the *self* field of *func3*. This is again followed up by *func1*.